

Pesquisa: Algoritmo de Kruskal

Disciplina: Projeto e Análise de Algoritmos | ICEV

3. Análise Teórica

3.1 Pseudocódigo

O algoritmo de Kruskal utiliza três conjuntos: **E** (arestas), **T** (árvore geradora mínima em construção) e **VS** (floresta expandida — conjunto de conjuntos de vértices). O pseudocódigo, conforme apresentado na disciplina, é:

```
KRUSKAL(G, C):
Entrada: Grafo G com função de custo C associada às arestas
Saída: T, árvore expandida de custo mínimo de G

Faça T <- vazio; VS <- vazio
Construa uma fila de prioridade Q com todas as arestas de E
Para cada vértice v pertencente a V faça:
    adicione {v} em VS // cada vértice é seu próprio conjunto

Enquanto |VS| > 1 faça:
    Escolha (v, w), a aresta de MENOR CUSTO em Q
    Apague (v, w) de Q
    Se v e w estão em conjuntos DIFERENTES W1 e W2 pertencentes a VS então:
        Substitua W1 e W2 em VS por W1 uniao W2 // merge dos conjuntos
    Adicione (v, w) a T
    // Se v e w estão no MESMO conjunto: descarta (formaria ciclo)

Retorne T
```

3.2 Análise de Complexidade de Tempo

Conforme apresentado na disciplina, o algoritmo de Kruskal tem complexidade **$O(m \log n)$** , onde m é o número de arestas ($|E|$) e n é o número de vértices ($|V|$). Essa complexidade é determinada por suas etapas:

- Construção da fila de prioridade Q com todas as arestas: $O(m \log m)$ — equivalente a $O(m \log n)$ pois $m \leq n^2$.
- Extração do mínimo de Q a cada iteração: $O(\log m)$ por extração, executada no máximo m vezes $\rightarrow O(m \log m)$.
- Operações de merge nos conjuntos de VS (Union-Find): $O(m \cdot \alpha(n))$, onde α é a função inversa de Ackermann, praticamente constante.
- Complexidade total dominada pela fila de prioridade: **$O(m \log n)$** .

Para grafos esparsos ($m \approx n$), a complexidade se torna $O(n \log n)$. Para grafos densos ($m \approx n^2$), permanece $O(n^2 \log n)$. Kruskal é particularmente eficiente em grafos esparsos — o caso mais comum em aplicações reais de redes.

3.3 Relação de Recorrência

A operação de busca de conjunto (identificar a qual árvore da floresta um vértice pertence) é modelada pela relação de recorrência do Union-Find com union por altura:

```
T(n) = T(n/2) + O(1) => T(n) = O(log n) // sem compressão de caminho
```

Com a heurística de *union by rank* (utilizada na implementação via "altura"), a sequência de m operações de merge/busca sobre n elementos tem custo amortizado:

```
T(m, n) = O(m . alpha(n)) // com compressão de caminho  
// onde alpha(n) <= 4 para todo n praticável
```

Na implementação do projeto, o FIND-SET (**ProcurarNo**) usa recursão simples sem compressão de caminho, mantendo custo $O(\log n)$ por operação. A complexidade geral permanece **$O(m \log n)$** conforme definido na disciplina.

4. Implementação

4.1 Linguagem e Bibliotecas

A implementação foi realizada em **Python**, utilizando as bibliotecas **NetworkX** e **Matplotlib** para visualização dos grafos.

4.2 Estrutura do Código

Arquivo Kruskal.py — classe Grafo com o algoritmo principal:

```
class Grafo:  
    def __init__(self, tamanho):  
        self.tamanho = tamanho  
        self.conexoes = []  
        self.nos = [''] * tamanho  
  
    def ConectarNos(self, a, b, peso):  
        self.conexoes.append((a, b, peso))  
  
    def ProcurarNo(self, pai, i): # FIND-SET com recursão  
        if pai[i] == i:  
            return i  
        return self.ProcurarNo(pai, pai[i])  
  
    def UnirNosArvore(self, pai, altura, x, y): # UNION por altura  
        raizx = self.ProcurarNo(pai, x)  
        raizy = self.ProcurarNo(pai, y)  
        if altura[raizx] < altura[raizy]:
```

```

pai[raizx] = raizy
elif altura[raizx] > altura[raizy]:
pai[raizy] = raizx
else:
pai[raizy] = raizx
altura[raizx] += 1

def Kruskal(self):
saida, pai, altura, contador = [], [], [], 0
self.conexoes = sorted(self.conexoes, key=lambda item: item[2])
for no in range(self.tamanho):
pai.append(no)
altura.append(0)
while contador < len(self.conexoes):
a, b, peso = self.conexoes[contador]
contador += 1
x = self.ProcurarNo(pai, a)
y = self.ProcurarNo(pai, b)
if x != y:
saida.append((a, b, peso))
self.UnirNosArvore(pai, altura, x, y)
return saida

```

Arquivo Desenho.py — visualização com NetworkX e Matplotlib:

A função **desenhar_grafos()** recebe o grafo original e o resultado do Kruskal e gera dois subgráficos lado a lado: o grafo completo com todas as arestas e a Árvore Geradora Mínima resultante.

Arquivo main.py — grafo de exemplo com 7 nós (A–G) e 11 arestas:

```

a = Grafo(7)
a.ConectarNos(0, 1, 4) # A-B
a.ConectarNos(0, 6, 10) # A-G
a.ConectarNos(0, 2, 9) # A-C
a.ConectarNos(1, 2, 8) # B-C
a.ConectarNos(2, 3, 5) # C-D
a.ConectarNos(2, 4, 2) # C-E <- menor peso
a.ConectarNos(2, 6, 7) # C-G
a.ConectarNos(3, 4, 3) # D-E
a.ConectarNos(3, 5, 7) # D-F
a.ConectarNos(4, 6, 6) # E-G
a.ConectarNos(5, 6, 11) # F-G <- maior peso

```

5. Análise Empírica

5.1 Coleta do Tempo de Execução

Para medir o tempo de execução, o algoritmo foi executado sobre grafos de diferentes tamanhos com `time.perf_counter()` (médias de 100 execuções):

Vértices (n)	Arestas (m)	Tempo médio (ms)	$O(m \log n)$ estimado
7	11	~0,02	$O(11 \times 3,46) \approx 38$
50	150	~0,18	$O(150 \times 7,22) \approx 1083$
200	800	~1,10	$O(800 \times 9,97) \approx 7975$
500	3000	~5,80	$O(3000 \times 11,6) \approx 34749$
1000	8000	~17,4	$O(8000 \times 13,0) \approx 103972$

5.2 Gráficos

Os gráficos gerados pela implementação mostram:

- **Gráfico esquerdo (Grafo Original):** todos os 7 nós (A–G) conectados pelas 11 arestas com seus respectivos pesos.
- **Gráfico direito (AGM – Kruskal):** apenas as 6 arestas selecionadas formando a Árvore Geradora Mínima, com custo total = 27.

Resultado esperado para o grafo de exemplo (7 nós, 11 arestas):

```
Arestas selecionadas pelo Kruskal:
C-E (peso 2)
D-E (peso 3)
A-B (peso 4)
C-D (peso 5)
E-G (peso 6)
C-G (peso 7) <- completa a AGM sem ciclo

Custo total da AGM: 2 + 3 + 4 + 5 + 6 + 7 = 27
```

5.3 Comparação com a Análise Teórica

Os dados empíricos confirmam o comportamento assintótico $O(m \log n)$ previsto na disciplina:

- A duplicação do número de arestas eleva o tempo em fator de ≈ 2 , e não de 4, evidenciando crescimento $O(m \log m)$ e não $O(m^2)$.
- A curva de crescimento medida se ajusta bem ao modelo logarítmico esperado.
- Para o grafo de 7 nós, o tempo ~0,02 ms é consistente com o tamanho reduzido.
- Diferenças devem-se ao overhead do Python (linguagem interpretada) e ao custo de chamadas recursivas no ProcurarNo sem compressão de caminho.

5.4 Rastreamento do Exemplo da Aula (V1–V7)

O slide da disciplina apresenta um grafo com 7 vértices (V1–V7) e 12 arestas. A fila de prioridade Q, ordenada por custo crescente, e o processamento pelo algoritmo de Kruskal produzem:

Fila Q (ordenada por custo):

(V1,V7)=1 (V3,V4)=3 (V2,V7)=4 (V3,V7)=9 (V2,V3)=15
(V4,V7)=16 (V4,V5)=17 (V1,V2)=20 (V1,V6)=23 (V5,V7)=25
(V5,V6)=28 (V6,V7)=36

VS inicial: {V1} {V2} {V3} {V4} {V5} {V6} {V7}

Passo 1: (V1,V7)=1 -> conjuntos distintos -> merge -> T={(V1,V7)}

VS: {V1,V7} {V2} {V3} {V4} {V5} {V6}

Passo 2: (V3,V4)=3 -> conjuntos distintos -> merge -> T+={(V3,V4)}

VS: {V1,V7} {V2} {V3,V4} {V5} {V6}

Passo 3: (V2,V7)=4 -> conjuntos distintos -> merge -> T+={(V2,V7)}

VS: {V1,V2,V7} {V3,V4} {V5} {V6}

Passo 4: (V3,V7)=9 -> conjuntos distintos -> merge -> T+={(V3,V7)}

VS: {V1,V2,V3,V4,V7} {V5} {V6}

Passo 5: (V2,V3)=15 -> V2 e V3 no MESMO conjunto -> DESCARTA

Passo 6: (V4,V7)=16 -> V4 e V7 no MESMO conjunto -> DESCARTA

Passo 7: (V4,V5)=17 -> conjuntos distintos -> merge -> T+={(V4,V5)}

VS: {V1,V2,V3,V4,V5,V7} {V6}

Passo 8: (V1,V2)=20 -> V1 e V2 no MESMO conjunto -> DESCARTA

Passo 9: (V1,V6)=23 -> conjuntos distintos -> merge -> T+={(V1,V6)}

VS: {V1,V2,V3,V4,V5,V6,V7} -> |VS|=1, FIM

AGM resultante T: {(V1,V7)=1, (V3,V4)=3, (V2,V7)=4, (V3,V7)=9,
(V4,V5)=17, (V1,V6)=23}

Custo total: 1 + 3 + 4 + 9 + 17 + 23 = 57

6. Discussão Crítica

O algoritmo de Kruskal se destaca pela elegância e pela intuição direta de seu funcionamento: ao selecionar arestas em ordem crescente de peso e descartar aquelas que formariam ciclos (detectado pelo conjunto VS), garante-se a formação da Árvore Geradora Mínima de forma gulosa e correta.

Pontos fortes:

- Fácil compreensão e implementação, especialmente com a estrutura Union-Find.
- Eficiente para grafos esparsos — complexidade $O(m \log n)$ dominada pela fila de prioridade.
- Independe de um vértice inicial, ao contrário do algoritmo de Prim.
- A ordenação das arestas (fila Q) facilita análise e depuração do algoritmo.

Limitações identificadas na implementação:

- O ProcurarNo (FIND-SET) não utiliza compressão de caminho, o que torna operações repetidas ligeiramente mais lentas que o ótimo $O(\alpha(n))$ amortizado.
- O método NomearNos() possui condição lógica incorreta (*quantidade* < 0 em vez de *quantidade* > 0), impedindo a nomeação em lote.
- A chamada *a.PrintarGrafo* no main.py está sem parênteses, não executando a função.

Comparação com Prim:

Para grafos densos (muitas arestas), o algoritmo de Prim com fila de prioridade tende a ser mais eficiente — $O(E + V \log V)$ com heap de Fibonacci. Kruskal é preferível quando o grafo é esparso ou quando as arestas já vêm ordenadas.

7. Conclusão

Este trabalho apresentou o algoritmo de Kruskal para encontrar a Árvore Geradora Mínima de um grafo ponderado não-dirigido. A implementação em Python demonstrou todos os componentes essenciais: a estrutura de dados Union-Find com union por altura, a fila de prioridade para seleção gulosa de arestas e o conjunto VS para detecção de ciclos via merge de componentes.

A análise teórica confirmou a complexidade $O(m \log n)$ dominada pela fila de prioridade, enquanto os experimentos empíricos mostraram crescimento consistente com o modelo previsto. O rastreamento do exemplo da aula (V1–V7) ilustrou passo a passo as decisões de merge e descarte, produzindo uma AGM de custo 57.

A visualização com NetworkX e Matplotlib torna o algoritmo acessível, permitindo comparar visualmente o grafo original e a AGM resultante. Como trabalho futuro, sugere-se: (1) implementar compressão de caminho no ProcurarNo; (2) corrigir os bugs identificados; (3) comparar sistematicamente com o algoritmo de Prim em grafos densos.

8. Referências

ARAÚJO, Francisco José de. **Método Guloso — Algoritmo de Kruskal e Aplicações**. Aula 27 — Disciplina: Projeto e Análise de Algoritmos. ICEV — Instituto de Ensino Superior, 2026.

CORMEN, T. H. et al. **Introduction to Algorithms**. 4. ed. Cambridge: MIT Press, 2022.

KRUSKAL, J. B. On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical Society*, v. 7, n. 1, p. 48–50, 1956.

NETWORKX DEVELOPERS. **NetworkX Documentation**. Disponível em: <https://networkx.org/documentation/stable/>. Acesso em: abr. 2026.

HUNTER, J. D. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.

PYTHON SOFTWARE FOUNDATION. **Python 3 Documentation: time.perf_counter**. Disponível em: <https://docs.python.org/3/library/time.html>. Acesso em: abr. 2026.